HELIXOR SOLVER EXAMPLE · GUIDE

Ocean container routing, explained

An importer has 232 containers to move from three Chinese ports to two US ports over six weeks, on four carriers. This guide walks through the business problems that planning raises, shows each one as a scenario, and explains how the hosted Helixor Solver handles it. Every number is measured, and you can reproduce it from this repository.

Repository: github.com/HelixorAI/helixor-solver-ocean-container-routing

---

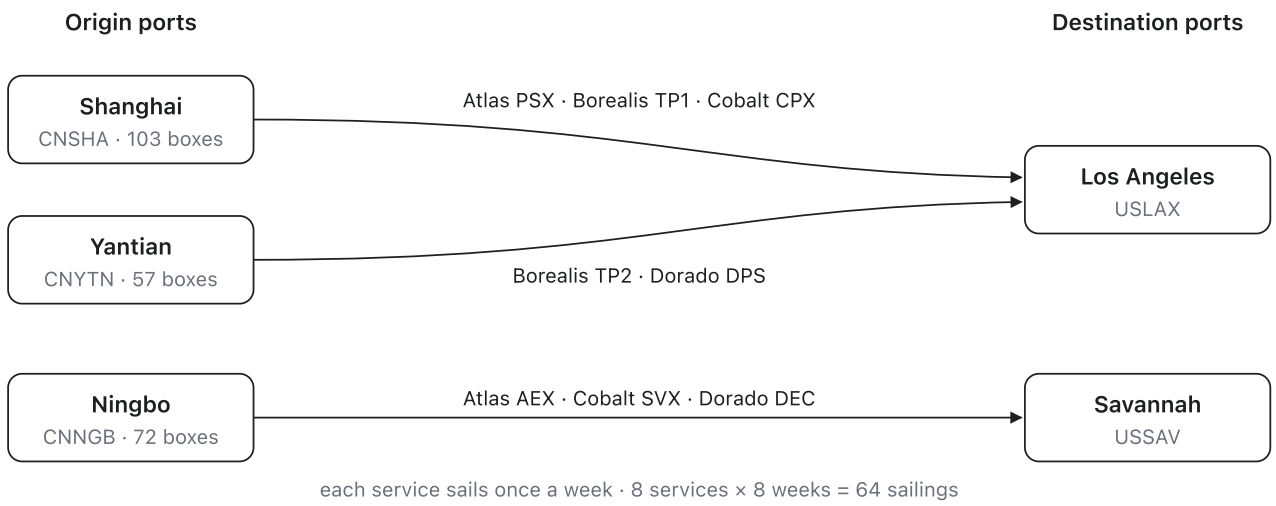
Contents

1. The business problem
2. What a good plan means
3. How the problem is modelled
4. How a plan is made and trusted
5. Scenario: the first plan
6. Scenario: taking over another system's plan
7. Scenario: the week goes wrong
8. Scenario: deciding what may change
9. Scenario: asking carriers for more space
10. Scenario: a peak-season surge
11. How good are the plans?
12. Run it yourself
13. Glossary

---

1. The business problem

An importer buys goods from vendors in China and ships them to the US in containers. It doesn't own ships. It books space on **sailings**: one voyage of one carrier's service, leaving on a fixed day each week. In this example, four carriers run eight weekly services on three **lanes** (a lane is a port pair).



The network. Each arrow is a lane, labelled with the carrier services that sail it. A container can only sail on its own lane. The carriers and vendors are invented; the ports are real.

Four things make the booking hard:

Contracted space

Each carrier grants a weekly **allocation** on each sailing, for example 6 FEU. The importer has committed to it, so space it doesn't fill is money lost.

Spot space

Beyond the allocation a carrier may sell extra **spot** space at a higher rate, but only up to a limit, and only if it agrees.

Promise dates

Each container has a date its customer was promised. A sailing that arrives after it makes the container late.

Constant change

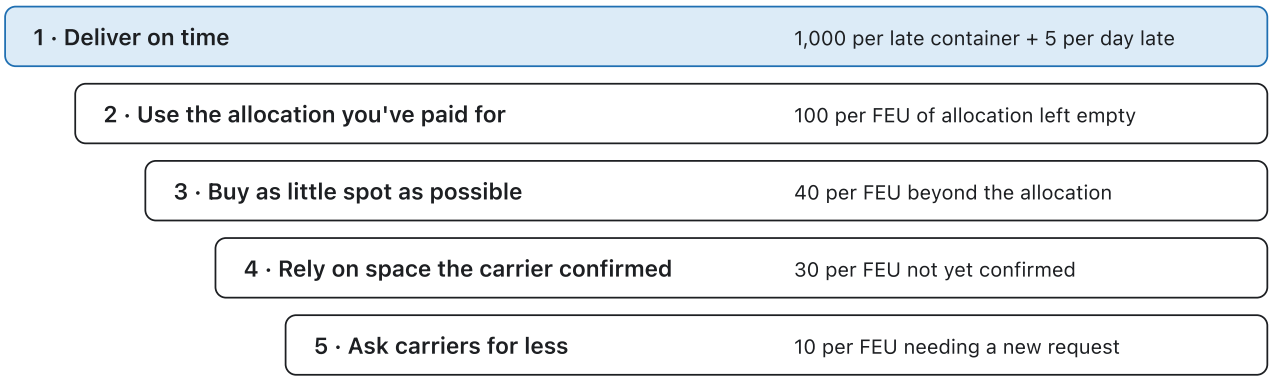
Sailings are blanked or delayed, cargo is late, vendors add orders, carriers cut allocations or refuse extra space. The plan has to be repaired without re-booking everything.

Sizes are counted in **FEU** (forty-foot equivalent units): a 40-foot container is 1 FEU, a 20-foot container is 0.5 FEU. The example has 155 40' high-cubes, 39 40' standard containers and 38 20' containers, and 224 FEU of allocation on the sailings of weeks 0 to 6.

---

2. What a good plan means

The business ranks its goals. The solver needs them as prices in one currency, so each goal becomes a weight per unit of shortfall. A higher goal gets a weight large enough that no amount of a lower goal can outweigh it in practice.



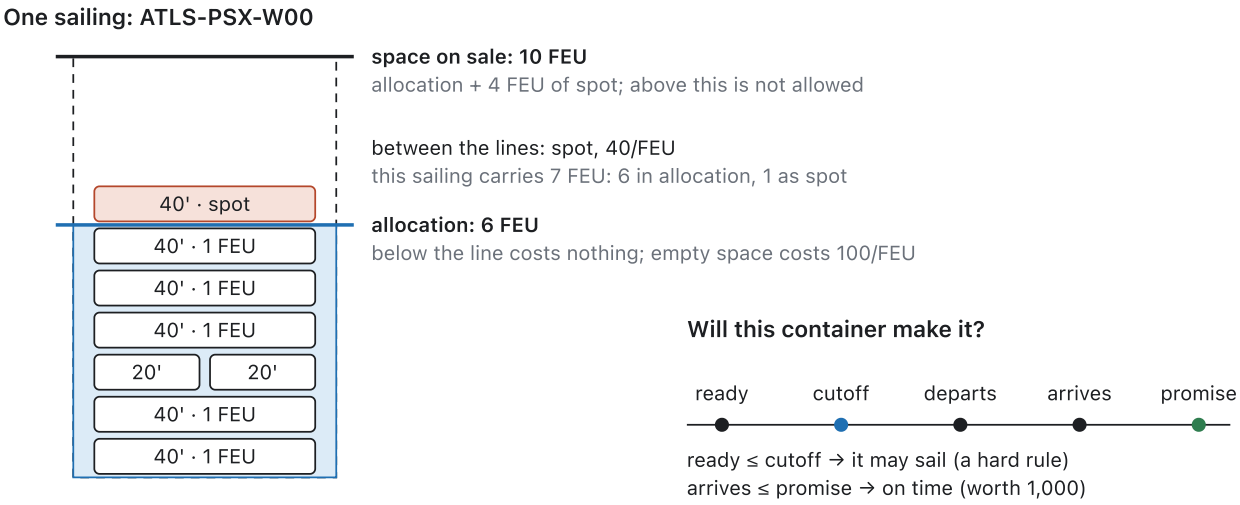
The objective. Lower in the list means cheaper, so the solver gives it up first. The numbers are the weights in the pack's manifest; a test checks that the independent scorer uses the same ones.

Why 100 for unused allocation and only 40 for spot? Allocation is paid for whether used or not, so leaving it empty and buying spot elsewhere is the worst of both. If your contracts work the other way, swap them; the weights are configuration, not code.

---

3. How the problem is modelled

The decision is simple to state: **for each container, pick one sailing**. It is bin packing (sailings are bins of limited space) combined with scheduling (each bin leaves and arrives on a fixed day).

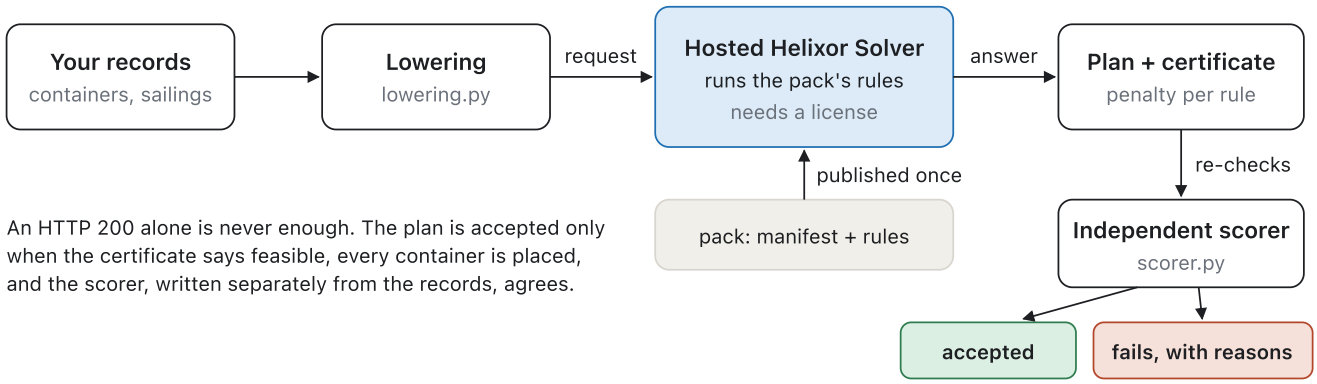


Left: a sailing is a bin with two lines. Filling up to the allocation is free; leaving it empty is the second-most expensive thing a plan can do. Space above the allocation is spot, up to the limit on sale. Right: a container can use a sailing only if its cargo is ready by the sailing's cutoff, and it is on time only if the sailing arrives by its promise date.

All of this is declared, not programmed. The **solver pack** (`pack/`) is a zip of configuration: a manifest naming the decision (containers onto sailings), eight rules, and their weights. Each rule uses a generic contract, such as "an attribute of the container equals an attribute of the sailing" for the lane rule, or "the

sum of FEU on a sailing stays below its capacity" for space. The solver has no ocean-shipping code. Your side does the **lowering**: it turns your records into the request, for example by working out, for each container, which sailings arrive on time.

4. How a plan is made and trusted



The trust path. The **certificate** is the solver's own account of the plan: feasible or not, and the penalty of each rule. The **independent scorer** reimplements the business rules directly from your records without the solver. If they disagree, the command stops; it never passes on a plan it couldn't verify.

5. Scenario: the first plan

> **BUSINESS**
> It's Monday of week 0. All 232 containers need a sailing for the next six weeks.

> **SOLVER**
> One request with every container and sailing, no prior plan. It searches for the cheapest assignment that breaks no hard rule.

> **YOU SEE**
> A feasible plan: 193 of 232 on time, 92% of the allocation used, 6.5–7.5 FEU of spot.

Engine	Penalty	Late (days)	Allocation used	Spot FEU	Time
Exact MIP (proven optimum)	42,740	39 (294)	92.2%	6.5	0.2 s
Helixor solver (next release)	42,920	39 (294)	91.7%	7.5	4.5 s

Both deliver the same 193 containers on time, the top goal. The Helixor plan is 0.4% above the proven optimum on total penalty. Why are 39 containers late in every plan? No sailing on their lane arrives before their promise date, so no plan can save them; the solver still chooses the sailing that makes them least late.

6. Scenario: taking over another system's plan

BUSINESS

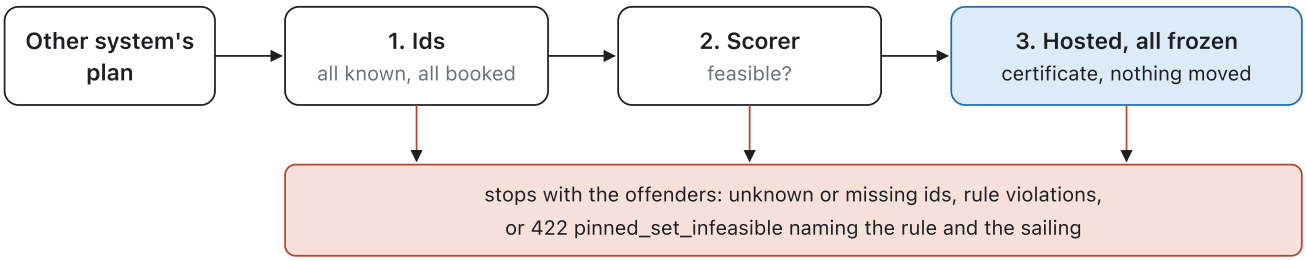
The importer already plans with another tool. It wants Helixor to take over from that plan, not start again.

SOLVER

The plan goes in with every booking *frozen*. The solver judges it under the pack's rules and moves nothing.

YOU SEE

The plan certified at the same penalty the scorer gives (42,780), or a refusal that names the broken rule and sailing.

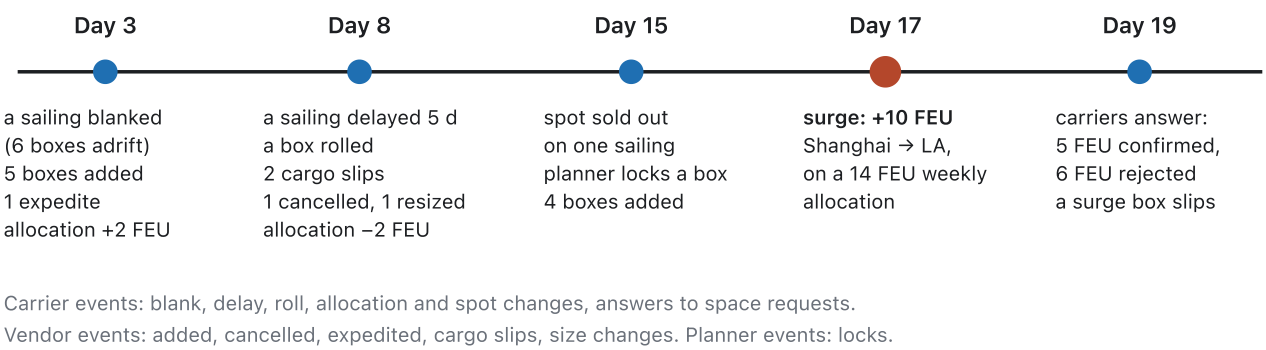


Adoption never quietly fixes a broken plan. If the frozen bookings alone break a hard rule (wrong lane, cargo not ready, a sailing over capacity), the solver refuses and says which. Once adopted, the plan is the starting point for everything that follows.

Bring the plan as `{"bookings": [{"container": …, "sailing": …}]}` or `{"placements": [{"entity": …, "group": …}]}`. The example adopts the exact MIP's plan, standing in for the other system.

7. Scenario: the week goes wrong

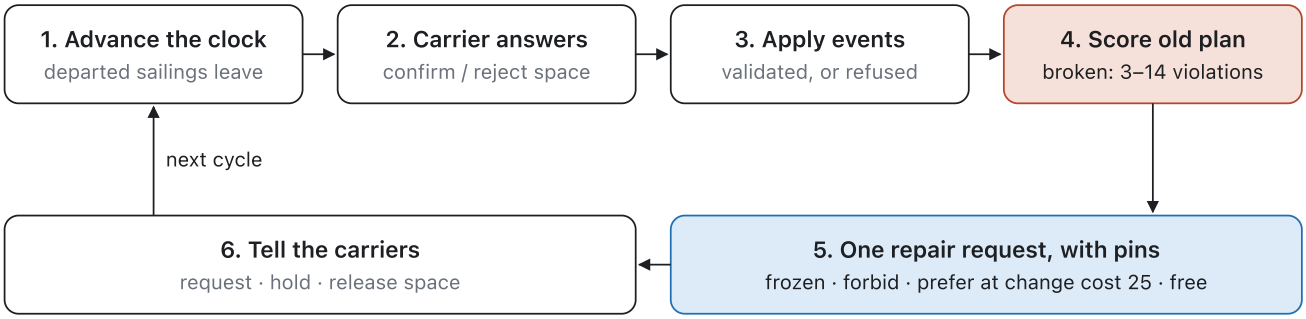
Plans don't survive the week. Every few days the planner rolls the clock forward, applies what happened, and asks for a **repair**: a new plan that fixes what broke and moves as little else as possible. The example replays five cycles over the first three weeks.



Carrier events: blank, delay, roll, allocation and spot changes, answers to space requests.
Vendor events: added, cancelled, expedited, cargo slips, size changes. Planner events: locks.

The replay in `data/events/replay.json`. Each event is validated; one that names an unknown sailing or container is refused, not skipped.

One cycle, step by step



Every repair in the replay comes back feasible, and the independent scorer confirms it.

The loop the `continuous` command runs. The old plan is always broken when an event lands; the repair makes it valid again.

Which containers may move?

Moving a container means re-booking it with a carrier, so the repair request carries a **pin** for every container. The replan policy chooses them:

Container	Pin	Why
Locked by a planner	`frozen`	an explicit decision
Sailing's cutoff within 7 days, in confirmed space	`frozen`	already booked with the carrier
Rolled off a sailing	`forbid` that sailing	the carrier won't take it
Touched by an event (blank, delay, slip, expedite, new)	none (free)	must be re-planned
Everything else	`prefer`, change cost 25	stays unless moving gains more than 25

A change cost of 25 means: move a container if that saves a late delivery (1,000) or fills a FEU of empty allocation (100), but not to shave four days off a container that is late anyway (4 × 5 = 20).

8. Scenario: deciding what may change

BUSINESS

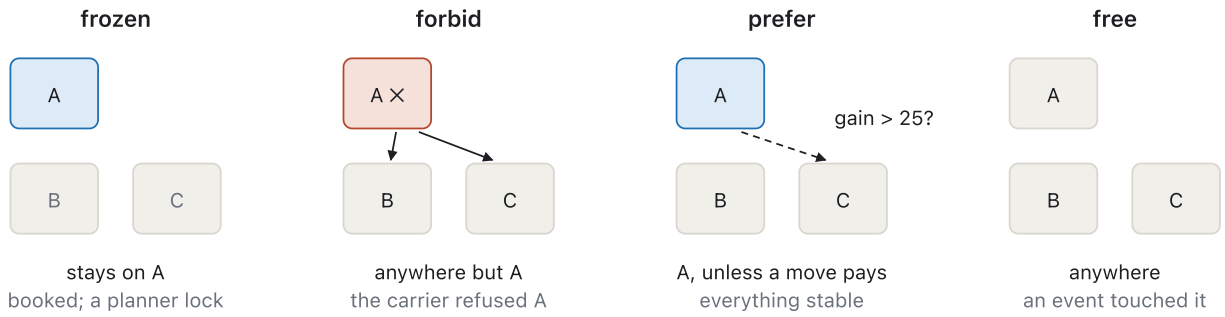
A carrier blanks one sailing (BORS-TP1-W01), and 5 containers lose their booking. How much of the rest of the plan should move?

SOLVER

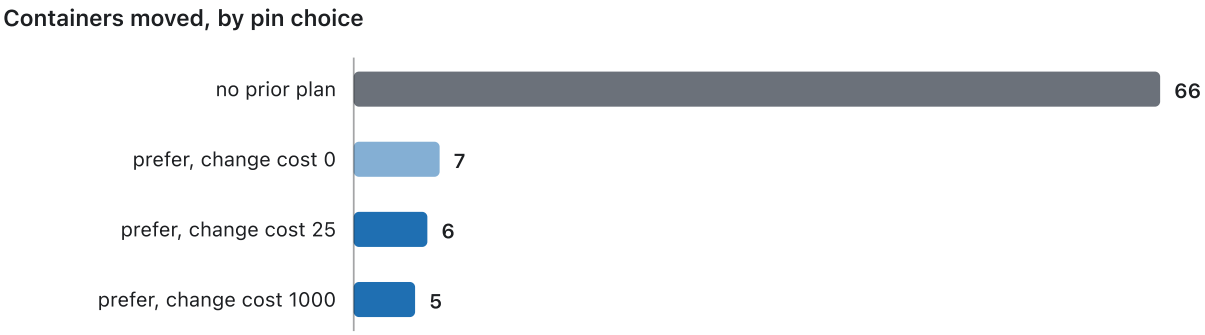
The same repair, run once per pin choice. Every pin is checked against the plan that comes back.

YOU SEE

The change cost works as a churn dial, and impossible pins are refused before any search.



The four pin modes for a container planned on sailing A. Hard rules still apply in every mode: a container can never go to a sailing on another lane or after its cutoff.



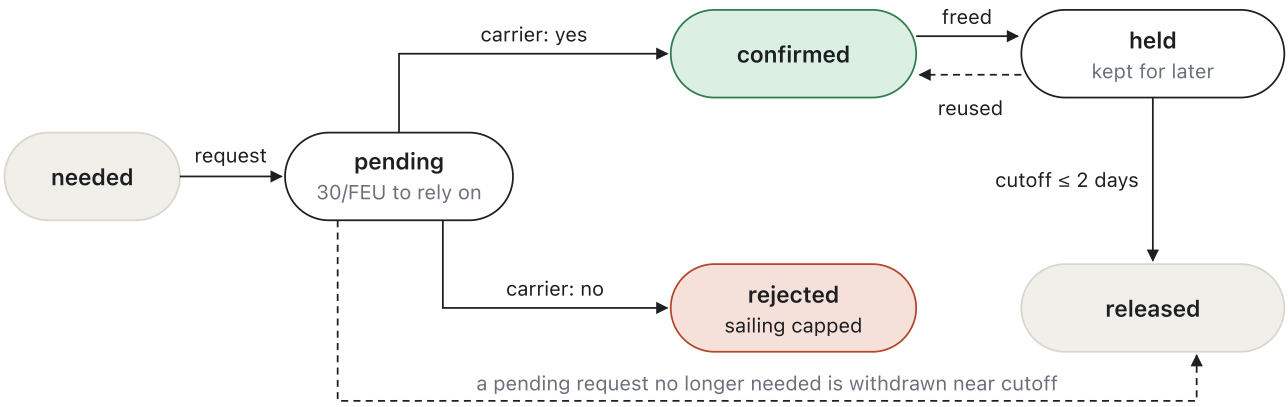
At 1000 only the 5 containers the blank displaced move, and the plan matches the optimum for those pins (44,115).

The change cost is the churn dial. Re-planning from scratch moves 66 containers. Keeping the prior plan as a preference moves 6–7 and scores better (44,025); at 1000 only the 5 that had to move do, for 90 more penalty.

Pins	Outcome	Penalty	Moved	Optimum, same pins
no prior plan	feasible	44,105	66	43,985
all prefer, change cost 0	feasible	44,025	7	43,985
all prefer, change cost 25	feasible	44,025	6	44,025
all prefer, change cost 1000	feasible	44,115	5	44,115
19 week-0 bookings frozen	feasible, pins held	44,045	6	44,045
3 containers forbidden COBT-CPX-W01	feasible, pins held	44,045	10	44,025
11 FEU frozen onto ATLS-PSX-W01 (10 FEU on sale)	**refused**: 422 `pinned_set_infeasible`, sailing_capacity in ATLS-PSX-W01	–	–	–

9. Scenario: asking carriers for more space

The allocation is a contract, so the carrier has already agreed to it. Anything beyond it is a request. The importer can **over-allocate**, asking for more than the allocation and seeing if the carrier accepts, often through the carrier's booking website. The carrier may say no. And when space is freed up, a planner doesn't necessarily hand it back, because a later change may need it.



How each FEU beyond the allocation moves through its life. The solver prices the uncertain states (30 per FEU not yet confirmed, 10 more per FEU needing a new request), so it prefers held space, then space already asked for, then a new request. It still goes beyond the allocation whenever that delivers a container on time.

BUSINESS

A booking on unconfirmed space isn't really booked. If the carrier says no, those containers must move.

SOLVER

Each sailing carries `confirmed_feu` and `committed_feu` (confirmed plus pending). After every plan, the planner works out requests, holds and releases. A rejection lowers the sailing's capacity, and the next repair moves what no longer fits.

YOU SEE

Lines like `carrier action: request 2 FEU on ATLS-PSX-W05` and `BORS-TP1-W03: carrier REJECTED 1 FEU; space now capped at 5 FEU`.

Two details follow from the business rule. The replan policy freezes a booking near cutoff only within confirmed space, because a booking on pending space isn't with the carrier yet; the run says so in a note. And in the replay, a simulator plays the carriers: the scenario's hidden `simulation` block says how much extra each carrier will really take. The planner never reads it and it is never sent to the solver; a test checks this. Plans are made on what carriers have said, not on what they will say.

In the replay, after cycle 1 the plan holds 0.5 FEU of confirmed space it no longer needs on BORS-TP2-W02. It keeps it through cycle 2 in case a later change needs it, and releases it in cycle 3, once that sailing's cutoff is within two days.

10. Scenario: a peak-season surge

BUSINESS

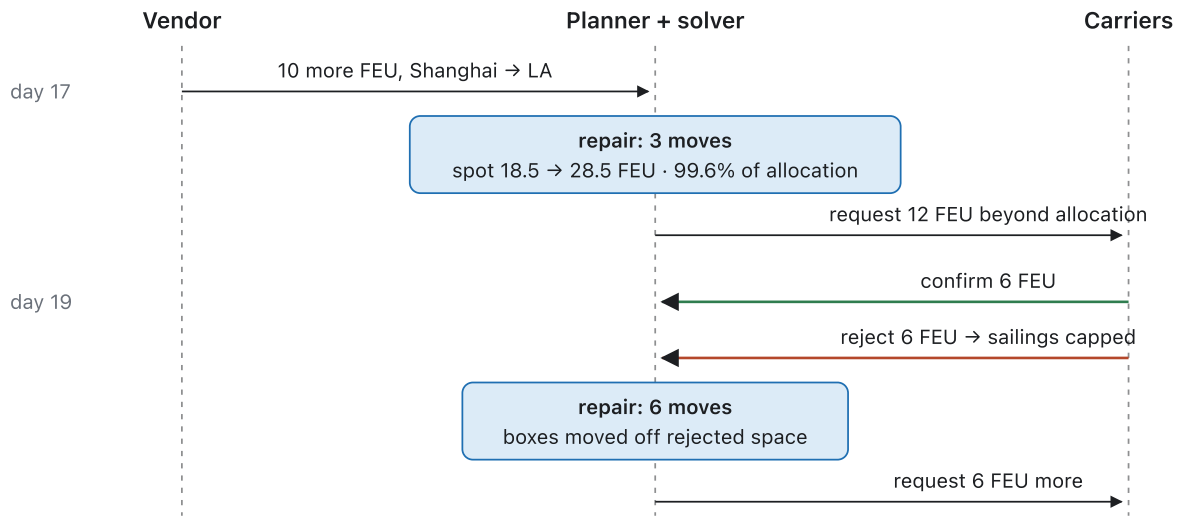
On day 17 a vendor adds 10 FEU for Shanghai to Los Angeles. The lane's weekly allocation is 14 FEU and already nearly full. The demand can't fit in the contract.

SOLVER

It fills the allocation and buys spot for the rest, choosing sailings that keep the most containers on time. The planner requests the extra space.

YOU SEE

Spot rises from 18.5 to 28.5 FEU with 99.6% of the allocation used; then carriers reject 6 of 12 FEU, and the next repair re-routes.



Over-demand that needs spot, and what happens when carriers say no. The rejected FEU become hard capacity limits; the containers on them break the plan, and the next repair moves them to space that exists.

11. How good are the plans?

An **exact MIP** gets the exact same request as the Helixor solver and proves the optimum. A from-scratch hosted solve shows what re-planning everything would move.

Cycle	Helixor repair	Optimum, same pins	From scratch
1 · day 3	45,160 · 16 moved	45,160 · 13	45,220 · 72
2 · day 8	50,040 · 14	50,005 · 13	48,790 · 72
3 · day 15	46,825 · 5	46,825 · 2	43,705 · 60
4 · day 17 (surge)	54,825 · 3	54,825 · 3	46,890 · 54
5 · day 19	53,740 · 6	53,740 · 6	45,815 · 77

Penalty excludes the change cost; "moved" counts containers whose sailing changed from the previous plan.

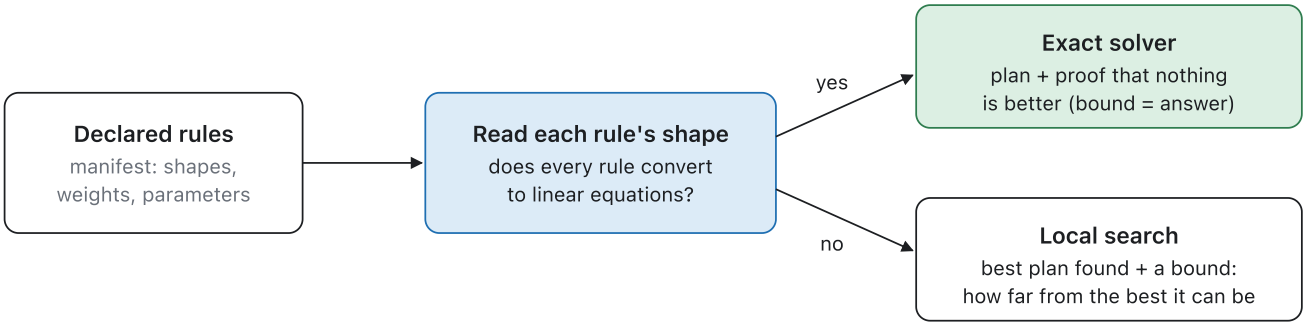
- **Every repair is feasible**, confirmed by the independent scorer.
- **The Helixor repair reaches the proven same-pins optimum in cycles 1, 3, 4 and 5**, and is 0.07% above it in cycle 2 (35 points), in 2.1–4.1 seconds.
- **Repair moves far less than re-planning:** 3–16 containers against 54–77. Where it moves a few more than the optimum, the extra moves are free containers (ones an event touched), which carry no change cost, so they leave the score unchanged.
- **From scratch scores lower from cycle 2 on** because nothing holds it to the bookings already with carriers. It reaches that by moving 54–77 containers, frozen bookings included, which is why it isn't a plan anyone would send.

> **Measured on the next solver release.** These figures come from the next Helixor solver release, run on a local server with the same requests; the hosted service gets it on its next deploy. That release closed a gap the hosted repair had in these cycles (4.5–10% above the same-pins optimum) with two generic search changes: each scoring level now judges a worse move in its own units, and every solve ends with a pass that proves no single move or swap improves the plan. Neither change knows about containers or sailings.

Why a declared model can be proven optimal

The exact MIP in this example (`ocean_routing/optimum.py`) is not a second model of the business. It restates the manifest's own rules as equations, rule for rule, and it proves the optimum of every cycle in 0.02–0.3 seconds. That is possible because each rule in the manifest has a declared shape, and every one of those shapes can be written as linear equations:

Rule	Declared shape	As equations
lane match, cargo ready by cutoff	which sailings a container may use	the container's choice is restricted to those sailings
sailing capacity	a sum over a sailing, at most a limit	one inequality per sailing
on-time delivery, days late	a cost for each container–sailing pair	a coefficient on that choice
allocation, spot, unconfirmed and new space	a cost of a sailing's load above or below a line	one extra variable and one inequality per line



one model: the same declared rules, weights and configuration drive both

What a declared model makes possible. Because the solver can read what each rule is, not only what it scores, it can choose a method that proves its answer when the rules allow it, and say how close to the best its answer is when they don't.

What follows from the model being declared rather than programmed:

- **A proven answer where the rules allow one.** When every rule converts, the answer can come with a proof that no better plan exists, not just a good plan.
- **A measured gap where they don't.** Converting the rules that do convert gives a bound, so a plan found by search can say how far from the best possible it can be.
- **One model to maintain.** The equations come from the manifest, so a change to a rule, a weight or a customer's configuration reaches the exact method and the search at once. Nobody keeps a second copy in step.
- **Every answer stays checkable.** Whichever method produced the plan, the certificate scores it against the same declared rules, and the independent scorer re-checks it.

> **Status.** In this example the MIP is written by hand from the manifest, as a yardstick. Generating it from the manifest, and choosing between exact solving and search per request, is planned and not yet part of the hosted solver. Rules about sequences and timing along a route, and very large instances, will stay with search.

---

12. Run it yourself

Hosted solves need a Helixor account at helixor.dev whose organization holds a **Helixor Solver license**, plus the Solver API origin and a token for your account. The repository itself contains no solver. Everything marked `offline` runs without a license.

```
git clone https://github.com/HelixorAI/helixor-solver-ocean-container-routing.git
cd helixor-solver-ocean-container-routing
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest -q                                  # offline tests; live tests run only with
SOLVER_LIVE=1
```

Step	Command	Needs	About
Proven optimum	`python -m ocean_routing optimum`	offline	1 s
Score any plan	`python -m ocean_routing score out/plan-optimum.json`	offline	1 s
Replay, MIP repairs	`python -m ocean_routing continuous --start out/plan-optimum.json --offline` writes out/continuous-offline/	offline	5 s
Report	`python -m ocean_routing report out/plan-optimum.json -o out/report.html`	offline	1 s
Publish the pack	`python -m ocean_routing publish-pack`	license	5 s
First plan	`python -m ocean_routing plan`	license	1 min
Adopt a plan	`python -m ocean_routing adopt out/plan-optimum.json`	license	5 s
Hosted replay + comparisons	`python -m ocean_routing continuous --start out/plan-adopted.json --compare`	license	10 min
Pinning scenarios	`python -m ocean_routing pinning`	license	7 min

For hosted steps, copy `.env.example` to `.env`, fill in `SOLVER_URL` and `SOLVER_TOKEN`, and load it with `set -a && . ./.env && set +a`. Outputs land in `out/`. The README has the full command reference and every measured table; the tutorial explains each rule and file in depth. Source: https://github.com/HelixorAI/helixor-solver-ocean-container-routing.

13. Glossary

FEU	Forty-foot equivalent unit. A 40' container is 1 FEU, a 20' container 0.5.
Lane	An origin–destination port pair, such as CNSHA-USLAX.
Sailing	One weekly voyage of one carrier service, for example ATLS-PSX-W03: Atlas Line, service PSX, week 3.
Cutoff	The last day the container yard accepts cargo for a sailing.
Allocation	Weekly space a carrier has contracted to the importer on a sailing. Already acknowledged.
Spot	Space beyond the allocation, bought at market rates, up to what the carrier has on sale.
Promise date	The day the container's customer was promised delivery.
Solver pack	A versioned zip of configuration (manifest, rules, weights, a demo dataset) published to the hosted solver. Versions are immutable.
Lowering	Turning your records into the request: deriving on-time sailings, capacity, confirmed and committed space.
Certificate	The solver's account of a plan: feasible or not, and each rule's penalty.
Pin	Per-container instruction in a repair: `frozen`, `forbid`, `prefer`, or none.
Change cost	The price of moving a `prefer` container, in the same units as the weights.
Confirmed / committed	Confirmed: space the carrier acknowledged. Committed: confirmed plus pending requests.